

Знакомство с Intel[®] oneAPI (oneTBB)

Intel® Threading Building Blocks

Intel Threading Building Blocks (TBB) – это библиотека шаблонов для параллельного программирования на C++:

- ❑ Упрощает написание параллельных алгоритмов и скрывает явное управление потоками;
- ❑ Автоматически адаптируется к системе и позволяет достигнуть максимальной масштабируемости приложения;
- ❑ Поддерживает операционные системы :
 - Microsoft Windows
 - GNU/Linux
 - Apple Mac OS X
 - FreeBSD
 - Oracle Solaris
 - Community: QNX Neutrino, Xbox 360, Power PC

Ниша Intel® TBB

Облако/Датацентры Сопроцессоры Рабочие станции

Ноутбуки

Планшеты

Встроенное ПО



Кластеры

Серверы

Настольные
компьютеры

Нетбуки

Смартфоны

MPI

OpenMP

Intel® TBB

Структура Intel® TBB

Параллельные алгоритмы	Вычислительный граф (Flow Graph)	Конкурентные контейнеры		
Эффективная реализация типовых шаблонов параллелизма	Набор классов для описания вычислительных графов, которые могут исполняться параллельно	Контейнеры в стиле STL с возможностью конкурентного доступа без внешних блокировок		
		Примитивы синхронизации		
		Атомарные операции и различного рода мьютексы		
Планировщик задач		Локальная память потока	Потоки	Другое
Обеспечивает параллелизм на уровне задач, балансировка методом перехвата работы		Неограниченное число локальных переменных потока (TLS)	Обёртки для системных вызовов	Потокобезопасные таймеры и поддержка C++ исключений
Масштабируемый менеджер памяти				
Спроектирован для параллельных программ				

Параллельные алгоритмы и структуры данных	Потоки и синхронизация	Управление памятью и задачами
---	------------------------	-------------------------------

Состав Intel® Threading Building Blocks

- ❑ **Алгоритмы:** `parallel_for`, `parallel_reduce`, `parallel_invoke`, `parallel_sort`, `parallel_for_each`, `parallel_scan`, `parallel_pipeline`
- ❑ **Контейнеры:** `concurrent_queue`, `concurrent_vector`, `concurrent_hash_map`
- ❑ **Аллокаторы памяти:** `scalable_malloc`, `scalable_free`, `scalable_realloc`, `scalable_calloc`, `scalable_allocator`, `cache_aligned_allocator`
- ❑ **Мьютексы:** `mutex`, `spin_mutex`, `queuing_mutex`, `spin_rw_mutex`, `queuing_rw_mutex`, `recursive_mutex`
- ❑ **Таймеры, планировщик задач**

Особенности Intel® TBB

- ❑ Intel TBB позволяет абстрагироваться от низкоуровневых потоков и распараллеливать программу в терминах параллельно выполняющихся задач(task parallelism)
- ❑ Задачи TBB “легче” потоков операционной системы
- ❑ Планировщик TBB использует механизм "work stealing" для распределения задач по потокам
- ❑ Все компоненты Intel TBB определены в пространстве имен C++ (namespace) "tbb"

Intel® TBB Hello Word

```
#include <tbb/tbb.h>
using namespace tbb;

class MyTask {
public:
    MyTask(const char* name) : name_(name) {}
    void operator () () const {
        std::cout << "Hello from task " << name_ << std::endl;
    }
private:
    const char* name_;
};

int main()
{
    task_group tg;
    tg.run(MyTask("1")); /* Генерация задачи */
    tg.run(MyTask("2")); /* Генерация задачи */
    tg.wait(); /* Ожидание завершения задачи */
    return 0;
}
```

oneTBB task_group Class

task_group представляет собой одновременное выполнение группы задач. Задачи могут динамически добавляться в группу по мере ее выполнения. Структура класса:

```
class task_group {
public:
    task_group();
    ~task_group();

    // До C++11
    template<typename Func>
    void run(const Func& f);

    // Начиная с C++11
    template<typename Func>
    void run(Func&& f);

    template<typename Func>
    void run(task_handle<Func>& handle);

    template<typename Func>
    task_group_status run_and_wait(const Func& f);

    template<typename Func>
    task_group_status run_and_wait(task_handle<Func>& handle);

    task_group_status wait();
    bool is_canceling();
    void cancel();
};
```

oneTBB task_group Class

Пример

```
int Fib(int n) {  
    if (n < 2) {  
        return n;  
    }  
    else  
    {  
        int x, y;  
        tbb::task_group g;  
        // Генерация задачи как лямбда выражения  
        g.run([&] {x = Fib(n - 1); });  
        // Генерация задачи как лямбда выражения  
        g.run([&] {y = Fib(n - 2); });  
        // Ожидание результатов двух задач  
        g.wait();  
        return x + y;  
    }  
}
```

Глобальное управление потоками

Запрос максимального количества потоков по умолчанию которые будут созданы в неявной форме или в явной **task_arena**.

```
#include <tbb/info.h> или #include <oneapi/tbb/info.h>  
int num_threads = tbb::info::default_concurrency();
```

Установка количество потоков TBB глобально

```
tbb::global_control  
    global_limit(tbb::global_control::max_allowed_parallelism,  
        int КОЛИЧЕСТВО_ПОТОКОВ);
```

Проверка доступного количества потоков TBB

```
int allowed_threads =  
    tbb::global_control::active_value(tbb::global_control::  
        max_allowed_parallelism);
```

Распараллеливание циклов

В TBB реализованы шаблоны параллельных алгоритмов

- ❑ `parallel_for`
- ❑ `parallel_reduce`
- ❑ `parallel_invoke`
- ❑ `parallel_sort`
- ❑ `parallel_for_each`
- ❑ `parallel_scan`
- ❑ `parallel_pipeline`

Понятие диапазона (Range)

Диапазон (Range) можно рекурсивно разделить на две части.

Подразделение осуществляется путем вызова конструктора разбиения.

Есть два типа конструкторов разбиения:

- Базовый конструктор разбиения. В этом конструкторе рекомендуется, чтобы разделение было примерно на равные части, но это не обязательно. Максимально равномерное разделение обычно дает наилучший параллелизм.
- Конструктор пропорционального разбиения. Этот конструктор является необязательным и может быть опущен вместе с переменной класса `isSplittableInProportion`. При использовании этого типа конструктора для достижения наилучших результатов следуйте заданной пропорции с округлением до ближайшего целого числа, если это необходимо.

Понятие диапазона (Range)

Типы моделей диапазона

- `blocked_range` – модель одномерного диапазона.
Заголовок `#include <tbb/blocked_range.h>`
Синтаксис `template<typename Value> class blocked_range;`
- `blocked_range2d` – модель двухмерного диапазона.
Заголовок `#include "tbb/blocked_range2d.h"`
Синтаксис `template<typename RowValue, typename ColValue> class blocked_range2d;`
- `blocked_range3d` – модель трехмерного диапазона.
Заголовок `#include "tbb/blocked_range3d.h"`
Синтаксис `template<typename PageValue, typename RowValue, typename ColValue> class blocked_range3d;`
- `blocked_rangeNd` – модель N-мерного диапазона.

Класс blocked_range

```
class blocked_range {
public:
    // Типы
    typedef size_t size_type;
    typedef Value const_iterator;
    // Конструкторы
    blocked_range(Value begin, Value end, size_type grainsize = 1);
    blocked_range(blocked_range& r, split);
    blocked_range(blocked_range& r, proportional_split& proportion);
    // Активация пропорционального деления
    static const bool isSplittable_in_proportion = true;
    // Емкость (capacity)
    size_type size() const;
    bool empty() const;
    // Параметры доступа
    size_type grainsize() const;
    bool isDivisible() const;
    // Итераторы
    const_iterator begin() const;
    const_iterator end() const;
};
```

oneTBB parallel_for

parallel_for позволяет разбить пространство итерации на блоки (chunks), которые обрабатываются разными потоками.

Заголовок `#include <tbb/parallel_for.h>`

Синтаксис

```
template<typename Index, typename Func>
Func parallel_for(Index first, Index_type last, const Func & f
[, partitioner[, task_group_context & group]]);
```

```
template<typename Index, typename Func>
Func parallel_for(Index first, Index_type last,
Index step, const Func& f
[, partitioner[, task_group_context & group]]);
```

```
template<typename Range, typename Body>
void parallel_for(const Range& range, const Body& body,
[, partitioner[, task_group_context & group]]);
```

Организация цикла `parallel_for(first, last, step, f)` представляет собой параллельное выполнение цикла:

```
for (auto i = first; i < last; i += step) f(i);
```

oneTBV parallel_for

Управление распределением работы (partitioner)

Рекурсивное деление на максимально возможную глубину

```
parallel_for( range, functor, simple_partitioner() );
```

Глубина деления подбирается динамически

```
parallel_for( range, functor, auto_partitioner() );
```

Деление запоминается и по возможности воспроизводится

```
affinity_partitioner affp;  
parallel_for( range, functor, affp);
```

!!! Класс **affinity_partitioner** запоминает какими потоками выполнялись предыдущие итерации и пытается распределять блоки итераций с учетом этой информации – последовательные блоки назначаются на один и тот же поток для эффективного использования кеш-памяти

oneTBB parallel_for

Пример заполнения одномерного массива на основе лямбда-выражения

```
void Fill_array(double* arr, int size)
{
    tbb::parallel_for(tbb::blocked_range<int>(0, size),
        [&](tbb::blocked_range<int> r)
        {
            for (int i = r.begin(); i < r.end(); i++)
                arr[i] = (cos(i) * sin(i / 2.0)) * i;
        });
}
```

oneTBB parallel_for

Пример заполнения одномерного массива на основе функционального объекта

```
class ArrayFiller
{
    double* arr_;
public:
    ArrayFiller(double* arr) : arr_(arr) {}
    void operator()(const tbb::blocked_range<size_t>& r) const {
        for (size_t i = r.begin(); i != r.end(); i++)
        {
            arr_[i] = (cos(i) * 10 + sin(i + 2.5)) * 10;
        }
    }
};

void Fill_array(double* arr, int size)
{
    tbb::parallel_for(tbb::blocked_range<size_t>(0, size),
        ArrayFiller(arr));
}
```

oneTBB parallel_for

Пример заполнения двумерного массива на основе лямбда-выражения

```
void Fill_2d_array(double** arr, int size1, int size2)
{
    tbb::parallel_for(tbb::blocked_range2d<int>(0, size1, 0,
size2),
    [&](tbb::blocked_range2d<int> r)
    {
        for (int i = r.rows().begin(); i < r.rows().end(); i++)
            for (int j = r.cols().begin(); j < r.cols().end(); j++)
            {
                arr[i][j] = (cos(i) * sin(j / 2.0));
            }
    });
}
```

oneTBB parallel_reduce

Шаблон функции, вычисляющий операцию Редукции по диапазону.

```
template<typename Range, typename Value, typename Func, typename Reduction>
Value parallel_reduce(const Range& range, const Value& identity, const Func&
func, const Reduction& reduction, partitioner, task_group_context& group);
template<typename Range, typename Value, typename Func, typename Reduction>
Value parallel_reduce(const Range& range, const Value& identity, const Func&
func, const Reduction& reduction, partitioner);
template<typename Range, typename Value, typename Func, typename Reduction>
Value parallel_reduce(const Range& range, const Value& identity, const Func&
func, const Reduction& reduction, task_group_context& group);
template<typename Range, typename Value, typename Func, typename Reduction>
Value parallel_reduce(const Range& range, const Value& identity, const Func&
func, const Reduction& reduction);

template<typename Range, typename Body>
void parallel_reduce(const Range& range, Body& body, partitioner,
task_group_context& group);
template<typename Range, typename Body>
void parallel_reduce(const Range& range, Body& body, partitioner);
template<typename Range, typename Body>
void parallel_reduce(const Range& range, Body& body, task_group_context&
group);
template<typename Range, typename Body>
void parallel_reduce(const Range& range, Body& body);
```

oneTBB parallel_reduce

Пример на основе структурного объекта

```
struct Sum {  
    double value;  
    Sum() : value(0) {}  
    Sum(Sum& s, tbb::split) { value = 0; }  
    void operator()(const tbb::blocked_range<double*>& r) {  
        double temp = value;  
        for (double* a = r.begin(); a != r.end(); ++a) {  
            temp += *a;  
        }  
        value = temp;  
    }  
    void join(Sum& rhs) { value += rhs.value; }  
};  
  
double ParallelSum1(double* array, size_t n) {  
    Sum total;  
    parallel_reduce(tbb::blocked_range<double*>(array, array + n),  
        total);  
    return total.value;  
}
```

oneTBB parallel_reduce

Пример на основе класса

```
class reduce_par {
public:
    double sum;
    void operator()(const tbb::blocked_range<size_t>& r)
    {
        double sum_local = sum;
        double* xloc = x_;
        size_t end = r.end();
        for (size_t i = r.begin(); i != end; ++i)
            sum_local += xloc[i];
        sum = sum_local;
    }
    // Splitting constructor: вызывается при порождении новой
задачи
    reduce_par(reduce_par& r, tbb::split) : sum(0.0), x_(r.x_) {}
    // Join: объединяет результаты двух задач (текущей и r)
    void join(const reduce_par& r) { sum += r.sum; }
    reduce_par(double* x) : sum(0.0), x_(x) {}
private:
    double* x_;
};
```

oneTBB parallel_reduce

Пример на основе класса (продолжение)

```
double ParallelSum2(double*& arr, int size)
{
    reduce_par r(arr);
    parallel_reduce(tbb::blocked_range<size_t>(0, size), r);
    return r.sum;
}
```

oneTBB parallel_reduce

Пример на основе лямбда выражения

```
double ParallelSum3(double*& arr, int& size)
{
    auto total = tbb::parallel_reduce(
        // Задается диапазон
        tbb::blocked_range<int>(0, size),
        // задается начальное значение
        0.0,
        /* определяется лямбда выражение с указанием переменных
        диапазона и редуктора*/
        [&](tbb::blocked_range<int> r, double running_total)
        {
            // организация цикла вычислений
            for (int i = r.begin(); i < r.end(); ++i) {
                running_total += arr[i];
            }
            return running_total;
        }, std::plus<double>());
    return total;
}
```

oneTBB parallel_reduce

Пример одновременного поиска MIN и MAX значений

```
class MinMaxCalc
{
private:
    const double* const ArrayA;
public:
    double MinValue; //Минимальное значение
    double MaxValue; //Максимальное значение

    void operator()(const tbb::blocked_range<size_t>& r)
    {
        const double* a = ArrayA;
        double val;
        for (size_t i = r.begin(); i != r.end(); ++i)
        {
            val = a[i];
            if (val < MinValue) MinValue = val;
            if (val > MaxValue) MaxValue = val;
        }
    }
}
```

oneTBB parallel_reduce

Пример одновременного поиска MIN и MAX значений

```
MinMaxCalc(MinMaxCalc& x, tbb::split) :  
    ArrayA(x.ArrayA), MinValue(DBL_MAX), MaxValue(-DBL_MAX) {}
```

```
void join(const MinMaxCalc& y)  
{  
    if (y.MinValue < MinValue) MinValue = y.MinValue;  
    if (y.MaxValue > MaxValue) MaxValue = y.MaxValue;  
}
```

```
MinMaxCalc(const double* A) :  
    ArrayA(A), //Инициализация ArrayA от A  
    MinValue(DBL_MAX), // Инициализация MinValue  
    MaxValue(-DBL_MAX) // Инициализация MaxValue  
{  
};
```

...

```
MinMaxCalc MMC(arr); //Создание и инициализация объекта
```

```
MinMaxCalc
```

```
tbb::parallel_reduce(tbb::blocked_range<size_t>(0, size), MMC);  
std::cout << "Минимальное значение = " << MMC.MinValue <<  
    "\nМаксимальное значение = " << MMC.MaxValue << std::endl;
```

oneTBV parallel_reduce

Пример одновременного поиска MIN и MAX значений на основе структур и лямбда-выражений

```
// Структура для хранения Минимума и максимума с методами
// инициализации
struct minmax_st
{
    double v_min;
    double v_max;
    minmax_st() { v_min=DBL_MAX; v_max=-DBL_MAX; };
    minmax_st(double val) :v_min(val), v_max(val) {};
};

// Оператор для редуccionного действия
struct minmax_join {
    minmax_st operator()(const minmax_st& _Left, const minmax_st&
_Right) const {
        minmax_st tmp(0);
        tmp.v_max = max(_Left.v_max, _Right.v_max);
        tmp.v_min = min(_Left.v_min, _Right.v_min);
        return tmp;
    }
};
```

oneTBB parallel_reduce

Пример одновременного поиска MIN и MAX значений на основе структур и лямбда-выражений

```
...
// пример использования
minmax_st total_mm = tbb::parallel_reduce(
    tbb::blocked_range<size_t>(0, size), // Задается диапазон
    minmax_st(), // задается начальное значение
    [&](tbb::blocked_range<size_t> r, minmax_st running_maxmin) /*
определяется лямбда выражение с указанием переменных диапазона и
редуктора*/
    { // организация цикла вычислений
        for (size_t i = r.begin(); i < r.end(); i++) {
            running_maxmin.v_max = max(running_maxmin.v_max, arr[i]);
            running_maxmin.v_min = min(running_maxmin.v_min, arr[i]);
        }
        return running_maxmin;
    }, minmax_join() // Редукционное действие
);
// Доступ к результату
std::cout << " Минимальное значение = " << total_mm.v_min <<
"\n Максимальное значение = " << total_mm.v_max;
```

oneTBB parallel_invoke

Сфера применения: Для небольшого predetermined количества задач

Для использования: `#include <tbb/parallel_invoke.h>`

Синтаксис

```
template<typename Func0, typename Func1>  
void parallel_invoke(const Func0& f0, const Func1& f1);
```

```
template<typename Func0, typename Func1, typename Func2>  
void parallel_invoke(const Func0& f0, const Func1& f1,  
                    const Func2& f2);
```

```
template<typename Func0, typename Func1, ..., typename Func9>  
void parallel_invoke(const Func0& f0, const Func1& f1, ...,  
                    const Func9& f9);
```

Может быть от 2 до 10 аргументов. Каждый аргумент должен иметь тип, для которого определен оператор (). Обычно аргументы являются либо объектами функций, либо указателями на функции. Возвращаемые значения игнорируются.

oneTBV parallel_invoke

Пример

В примере выполняется параллельное выполнение функций `f()`, `g()` и `h()`. **g** и **h** являются функциональными объектами, которые могут содержать локальное состояние.

```
void f();
extern void MyFunction(int);

class MyFunctor {
int arg;
public:
MyFunctor(int a) : arg(a) {}
void operator()() const {MyFunction(arg); }
};

void RunFunctionsInParallel() {
MyFunctor g(2);
MyFunctor h(3);
tbb::parallel_invoke(f, g, h);
}
```

oneTBB parallel_invoke

Пример

Пример, переписанный с использованием лямбда-выражений C ++ 11, которые генерируют объекты функций.

```
#include <tbb/parallel_invoke.h>

using namespace tbb;

void f();
extern void MyFunction(int);

void RunFunctionsInParallel() {
    tbb::parallel_invoke(f, [] {MyFunction(2); },
        [] {MyFunction(3); });
}
```

oneTBB parallel_sort

Сфера применения: упорядочивание последовательности элементов

Применяется алгоритм нестабильной сортировки с трудоемкостью $O(N \times \log(N))$

!! Алгоритм не гарантирует сохранения порядка следования элементов с одинаковыми значениями.

Для использования: `#include <tbb/parallel_sort.h>`

Синтаксис

```
template<typename RandomAccessIterator>
void parallel_sort(RandomAccessIterator begin,
                  RandomAccessIterator end);

template<typename RandomAccessIterator, typename Compare>
void parallel_sort(RandomAccessIterator begin,
                  RandomAccessIterator end, const Compare& comp);

template<typename Container>
void parallel_sort(Container& c);

template<typename Container>
void parallel_sort(Container& c, const Compare& comp);
```

oneTBB parallel_sort

Пример

```
#include <tbb/parallel_sort.h>

int main()
{
    int size = 10000;
    double* arr1 = new double[size];
    double* arr2 = new double[size];

    Fill_array(arr1, size);
    Fill_array(arr2, size);

    parallel_sort(arr1, arr1 + size);
    parallel_sort(arr2, arr2 + size, std::greater<double>());

    Print_array(arr1, size);
    Print_array(arr2, size);

    return 0;
}
```